

Modelado y Tecnologías de Integración de Sistemas

Práctica Individual

Práctica 1

Interoperabilidad de Servicios Web

Sistema de Gestión de Edificio Inteligente

Alumno:	Juan Ponce de León Ruiz
Correo:	juankyleonruiz@gmail.com
Fecha:	Abril de 2026

Memoria descriptiva de la Práctica 1 — MTIS

Índice

1. Introducción	3
2. Arquitectura del Sistema	3
2.1 Servicios SOAP	3
2.2 Servicios REST	4
2.3 Infraestructura de soporte	4
3. Tecnologías Utilizadas	4
4. Implementación de los Servicios SOAP	5
4.1 Enfoque contract-first y generación de código	5
4.2 Autenticación con WSKey	5
4.3 Patrón Holder en JAX-WS	6
4.4 Validaciones de documentos españoles	6
5. Implementación de los Servicios REST	7
5.1 Diseño de recursos y especificación OpenAPI	7
5.2 Respuestas estandarizadas	7
5.3 Servicio de notificaciones	8
6. Modelo de Datos	8
7. Pruebas	9
8. Dificultades Encontradas y Aprendizajes	9

9. Conclusiones

11

1. Introducción

Esta memoria documenta el desarrollo de la Práctica 1 de la asignatura **Modelado y Tecnologías de Integración de Sistemas (MTIS)**, cuyo objetivo es implementar un sistema distribuido de gestión de edificio inteligente haciendo uso de dos paradigmas de servicios web: **SOAP** y **REST**.

El dominio del problema se centra en la gestión de empleados, el control de acceso a salas y el seguimiento de presencia dentro de un edificio, integrando además un servicio de notificaciones por correo electrónico.

La práctica ha supuesto un primer contacto directo con la creación de servicios web interoperables desde cero: definición de contratos (WSDL y OpenAPI), implementación de la lógica de negocio y despliegue de múltiples servicios que comparten una misma base de datos relacional.

2. Arquitectura del Sistema

El sistema está compuesto por **ocho microservicios independientes**, divididos en dos grupos según el protocolo que implementan.

2.1 Servicios SOAP

Los servicios SOAP siguen un enfoque *contract-first*: primero se define el contrato WSDL y a partir de él se genera el código Java mediante el plugin de Apache CXF.

Tabla 1. Servicios SOAP del sistema

Servicio	Responsabilidad	Puerto
<code>servicio-validaciones</code>	Validación de NIF, NIE, NAF e IBAN	8081
<code>servicio-empleados</code>	CRUD de empleados	8082
<code>servicio-control-accesos</code>	Registro y consulta de accesos	8083
<code>servicio-control-presencia</code>	Seguimiento de presencia en salas	8084

2.2 Servicios REST

Los servicios REST se documentan mediante especificaciones OpenAPI 3.0.3 y exponen recursos a través de las operaciones HTTP estándar (GET, POST, PUT, DELETE).

Tabla 2. Servicios REST del sistema

Servicio	Responsabilidad	Puerto
<code>servicio-salas</code>	Gestión de salas del edificio	8085
<code>servicio-niveles</code>	Gestión de niveles de acceso	8086
<code>servicio-dispositivos</code>	Gestión de dispositivos físicos	8087
<code>servicio-notificaciones</code>	Envío de notificaciones por email	8088

2.3 Infraestructura de soporte

Además de los servicios, el proyecto incluye los siguientes elementos de apoyo:

- **Base de datos MySQL** (`edificio_inteligente`): almacena todas las entidades del dominio y las claves de autenticación (*wskeys*).
- **Mailpit**: servidor SMTP local (puerto 1025) con interfaz web (puerto 8025) para inspeccionar los correos generados por el servicio de notificaciones durante el desarrollo.
- **Cliente de pruebas** (`ClienteMain.java`): aplicación Java que ejercita todos los servicios de forma automática y secuencial.

3. Tecnologías Utilizadas

Tabla 3. Tecnologías y herramientas empleadas

Tecnología	Uso en el proyecto
Java 21	Lenguaje de implementación
Spring Boot 3.2.3	Framework base de todos los servicios
Apache CXF 4.0.3	Motor JAX-WS para servicios SOAP
Spring Data JPA	Acceso a base de datos (ORM)
MySQL 8	Base de datos relacional compartida
Spring Mail	Envío de notificaciones por email
Maven	Gestión de dependencias y build
Mailpit	Servidor SMTP local para desarrollo
SoapUI	Pruebas manuales de servicios SOAP
Postman	Pruebas manuales de servicios REST

4. Implementación de los Servicios SOAP

4.1 Enfoque contract-first y generación de código

Para los servicios SOAP se ha seguido el enfoque *contract-first*: el WSDL es el punto de partida, y el plugin `cxfr-codegen-plugin` de Maven genera automáticamente las interfaces y clases de datos en tiempo de compilación. Esto garantiza que la implementación cumple exactamente el contrato publicado, sin riesgo de divergencia entre la definición del servicio y el código.

La estructura de cada servicio SOAP es la siguiente:

```
servicio-X/  
├─ src/main/resources/wsdl/      <-- Contrato WSDL  
├─ src/main/java/.../config/    <-- Configuración del endpoint CXF  
├─ src/main/java/.../impl/      <-- Lógica de negocio (implementa PortType)  
├─ src/main/java/.../model/     <-- Entidades JPA  
└─ src/main/java/.../repository/ <-- Repositorios Spring Data
```

4.2 Autenticación con WSKey

Todos los servicios —tanto SOAP como REST— requieren una clave de servicio (*wskey*) que se valida contra la tabla `wskeys` de la base de datos. Si la clave no existe o no es válida, el servicio devuelve un error de autenticación sin procesar la petición. De este modo se impide el acceso no autorizado sin necesidad de un mecanismo de autenticación externo.

La clave de pruebas utilizada en el entorno de desarrollo es `PRACTICAL_KEY_2024`, insertada directamente en la base de datos mediante el script `schema.sql`.

4.3 Patrón Holder en JAX-WS

Una de las particularidades de la implementación SOAP es el uso del patrón `Holder<T>` de JAX-WS para los parámetros de salida. Dado que Java no permite devolver múltiples valores de forma nativa, los parámetros anotados con `@WebParam(mode = Mode.OUT)` se pasan como `Holder<T>`, y el framework los mapea automáticamente al XML de respuesta SOAP.

```
@Override
public void validarNIF(
    @WebParam(name = "wskey") String wskey,
    @WebParam(name = "nif") String nif,
    @WebParam(name = "resultado", mode = Mode.OUT)
        Holder<Boolean> resultado,
    @WebParam(name = "mensaje", mode = Mode.OUT)
        Holder<String> mensaje) {

    if (!wskeyRepository.existsByClave(wskey)) {
        resultado.value = false;
        mensaje.value = "WSKey no válida";
        return;
    }
    // ... lógica de validación
}
```

Fragmento: firma con Holder en ValidacionesImpl.java

4.4 Validaciones de documentos españoles

El servicio `servicio-validaciones` implementa algoritmos de verificación para cuatro tipos de documentos:

- **NIF:** 8 dígitos + 1 letra verificadora, calculada como el módulo 23 del número sobre la tabla `TRWAGMYFPDXBNJZSQVHLCKE`.

- **NIE:** letra inicial (X/Y/Z) + 7 dígitos + letra verificadora. Se sustituye X→0, Y→1, Z→2 y se aplica el mismo algoritmo que el NIF.
- **NAF:** 12 dígitos; los dos últimos son el resultado de calcular módulo 97 sobre los diez primeros dígitos.
- **IBAN:** 24 caracteres para el formato español (ES + 22 dígitos); verificación conforme a ISO 13616 mediante reordenamiento y módulo 97.

5. Implementación de los Servicios REST

5.1 Diseño de recursos y especificación OpenAPI

Los servicios REST exponen recursos siguiendo las convenciones HTTP. Cada servicio cuenta con su especificación OpenAPI 3.0.3 en formato YAML, que documenta todos los endpoints, parámetros, cuerpos de petición y respuestas posibles antes de comenzar la implementación Java.

Tabla 4. Endpoints de ejemplo — servicio-salas

Método	Ruta	Descripción
GET	/api/salas/{codigosala}	Obtener una sala por código
POST	/api/salas	Crear una nueva sala
PUT	/api/salas/{codigosala}	Actualizar los datos de una sala
DELETE	/api/salas/{codigosala}	Eliminar una sala

5.2 Respuestas estandarizadas

Todos los servicios REST devuelven un objeto JSON con los campos `codigo` y `mensaje`, además del recurso solicitado en las operaciones de consulta. Los códigos de estado HTTP usados son los estándar:

- `200 OK` — operación completada con éxito.
- `201 Created` — recurso creado.
- `400 Bad Request` — datos de entrada inválidos.
- `401 Unauthorized` — wskey ausente o inválida.
- `404 Not Found` — recurso no encontrado.
- `409 Conflict` — conflicto con un recurso existente (clave duplicada).

5.3 Servicio de notificaciones

El servicio `servicio-notificaciones` utiliza **Spring Mail** para enviar correos electrónicos en tres situaciones:

1. Validación exitosa de un usuario — `POST /api/notificaciones/usuario-valido/{nif}` .
2. Notificación de error — `POST /api/notificaciones/error` .
3. Cambio de presencia en una sala — `POST /api/notificaciones/presencia-sala` .

En el entorno de desarrollo se emplea Mailpit como servidor SMTP local, lo que permite verificar los correos sin necesidad de un servidor de correo real. La interfaz web de Mailpit (puerto 8025) muestra los mensajes recibidos en tiempo real.

6. Modelo de Datos

La base de datos MySQL `edificio_inteligente` contiene las siguientes tablas:

Tabla 5. Tablas de la base de datos

Tabla	Clave primaria	Descripción
<code>empleados</code>	<code>nif</code>	Datos de los empleados del edificio
<code>niveles</code>	<code>nivel</code> (entero)	Niveles de acceso (1=General, 2=Restringido, 3=Alta seguridad)
<code>salas</code>	<code>codigosala</code>	Salas del edificio, asociadas a un nivel de acceso
<code>dispositivos</code>	<code>codigodispositivo</code>	Dispositivos de control instalados en cada sala
<code>controlaccesos</code>	<code>id</code> (autoincremental)	Historial de eventos de acceso
<code>controlpresencia</code>	(<code>nif</code> , <code>codigosala</code>)	Presencia actual de empleados en salas
<code>wskeys</code>	<code>id</code> (autoincremental)	Claves de autenticación de los servicios web

La tabla `controlpresencia` utiliza una clave primaria compuesta (`nif`, `codigosala`) para garantizar que cada empleado no pueda aparecer registrado más de una vez en la misma sala de forma simultánea. A diferencia de `controlaccesos`, que es un registro histórico, `controlpresencia` refleja el estado en tiempo real.

7. Pruebas

7.1 Pruebas manuales

Las pruebas manuales se han realizado con las siguientes herramientas:

- **SoapUI** — servicios SOAP. Se construyen peticiones XML con la wskey correcta e incorrecta, verificando tanto el flujo nominal como los casos de error: NIF inválido, empleado inexistente, intento de duplicado, etc.
- **Postman** — servicios REST. Se cubren todos los verbos HTTP y se verifica que los códigos de estado de respuesta corresponden a los definidos en la especificación OpenAPI.
- **Mailpit** (interfaz web en `http://localhost:8025`) — para confirmar que los correos generados por el servicio de notificaciones llegan correctamente con el contenido esperado.

7.2 Cliente de pruebas automáticas

El módulo `cliente/` contiene la clase `ClienteMain.java`, que ejercita todos los servicios de forma secuencial. Para los servicios SOAP utiliza `JaxWsProxyFactoryBean` de Apache CXF; para los REST, el cliente HTTP nativo de Java (`java.net.http.HttpClient`, disponible desde Java 11).

8. Dificultades Encontradas y Aprendizajes

Esta práctica ha sido mi primer desarrollo completo de servicios web interoperables, lo que ha implicado un proceso de aprendizaje progresivo en varias áreas.

8.1 SOAP y JAX-WS

El principal obstáculo con SOAP fue comprender el modelo de programación de JAX-WS, en particular el patrón `Holder<T>` para parámetros de salida. Inicialmente intenté devolver los valores de respuesta directamente como resultado del método, lo que no funciona cuando el WSDL define múltiples parámetros de salida. Una vez entendido que JAX-WS mapea los parámetros *OUT* mediante `Holder`, la implementación resultó coherente con el contrato.

También fue necesario comprender la diferencia entre el enfoque *contract-first* (WSDL → código) y el enfoque *code-first* (código → WSDL), y por qué el primero es preferible en entornos de interoperabilidad real: el contrato es independiente del lenguaje de implementación y puede ser consumido por clientes escritos en cualquier plataforma.

8.2 REST y OpenAPI

La parte REST fue más accesible gracias a la familiaridad con Spring MVC. La principal lección fue la importancia de diseñar bien el contrato OpenAPI antes de implementar: los cambios posteriores en los endpoints obligan a actualizar tanto el código como la especificación, lo que genera trabajo adicional que se puede evitar con un buen diseño previo.

8.3 Configuración del entorno distribuido

La gestión de ocho servicios independientes, cada uno con su propio puerto y configuración, exigió atención en el orden de arranque —la base de datos debe estar disponible antes que cualquier servicio— y en la consistencia de las credenciales en todos los ficheros `application.properties`.

Un error concreto fue configurar el puerto SMTP a 25 en lugar de 1025 (el que utiliza Mailpit por defecto). Detectarlo requirió revisar los logs de Spring Mail, lo que fue una buena práctica para aprender a diagnosticar problemas en sistemas distribuidos.

8.4 Validaciones de documentos

Implementar los algoritmos de validación de NIF, NIE, NAF e IBAN desde cero supuso consultar la normativa española correspondiente y contrastar los resultados con validadores externos para asegurar la corrección de los cálculos. Fue un ejercicio útil para entender que los servicios de validación deben ser rigurosos: un error en el algoritmo podría rechazar documentos válidos o aceptar documentos inválidos.

8.5 Reflexión general

La práctica ha permitido comprender en la práctica por qué la interoperabilidad es un problema relevante en sistemas distribuidos: cuando los servicios están implementados con tecnologías distintas, el contrato (WSDL u OpenAPI) es el único elemento que garantiza que un cliente puede comunicarse con un servidor sin conocer sus detalles internos.

Haber trabajado con ambos paradigmas en paralelo ha hecho más clara la diferencia en su filosofía de diseño: SOAP orientado a operaciones con tipado fuerte y contratos explícitos, REST orientado a recursos con un modelo más simple y flexible. Cada uno tiene su contexto de aplicación, y conocer ambos permite elegir el más adecuado según los requisitos del sistema.

9. Conclusiones

La Práctica 1 ha resultado en un sistema funcional compuesto por ocho servicios web que cubren las necesidades de gestión de un edificio inteligente. Los objetivos planteados —implementar servi-

cios SOAP y REST, definir contratos previos a la implementación, compartir una base de datos relacional y añadir autenticación mediante wskey— se han cumplido en su totalidad.

Desde el punto de vista formativo, el desarrollo ha consolidado conceptos teóricos de la asignatura sobre interoperabilidad y ha introducido el uso de herramientas y frameworks habituales en el sector profesional: Apache CXF, Spring Boot, Spring Data JPA y OpenAPI. La experiencia de depurar un sistema distribuido con múltiples servicios también ha resultado formativa, ya que los errores pueden originarse en cualquier capa (red, configuración, lógica de negocio) y localizarlos requiere una metodología ordenada.

En definitiva, la práctica ha servido para trasladar los conceptos de interoperabilidad al plano práctico, con un sistema que, aunque está en un entorno local de desarrollo, refleja el tipo de arquitectura que se encuentra en sistemas reales de gestión empresarial.